

Schreiben und Lesen von Dateien in Java

Benjamin Malicke ©

11. März 2026

Inhaltsverzeichnis

1	Einführung: Dateien und Streams in Java	3
1.1	File-Objekt anlegen	3
1.2	Datei wirklich anlegen mit <code>createNewFile()</code>	3
2	Dateien schreiben mit <code>FileWriter</code>	3
2.1	Einfaches Schreiben in eine Datei	4
2.2	Datei anhängen statt überschreiben: <code>append</code>	4
2.3	Die Methoden <code>close()</code> und <code>flush()</code>	5
2.4	Vererbung: <code>FileWriter</code> und die Basisklasse <code>Writer</code>	5
3	Dateien schreiben mit <code>BufferedWriter</code>	5
3.1	Grundidee: Schreiben mit Puffer	6
3.2	Anhängen an eine Datei mit <code>BufferedWriter</code>	6
3.3	Puffer kontrollieren: <code>flush()</code> und <code>close()</code>	7
3.4	Vererbung und Zusammenspiel mit <code>Writer</code>	7
4	Dateien schreiben mit <code>PrintWriter</code>	8
4.1	Grundidee: Komfortable Textausgabe	8
4.2	Anhängen an eine Datei mit <code>PrintWriter</code>	8
4.3	Fehlerbehandlung mit <code>checkError()</code>	9
4.4	<code>flush()</code> und <code>close()</code> bei <code>PrintWriter</code>	9
4.5	Vererbung und Zusammenarbeit mit anderen <code>Writer</code> -Klassen	10
5	Vergleich: <code>FileWriter</code>, <code>BufferedWriter</code> und <code>PrintWriter</code>	10
5.1	Überblick in Tabellenform	11
5.2	Typischer Einsatz und Kombinationen	11
5.3	Vergleich wichtiger Methoden	11
5.4	Vererbung und Fehlerverhalten	11
6	Dateien lesen in Java – Einführung	12
7	Dateien lesen mit <code>FileReader</code>	12
7.1	Einfaches Lesen Zeichen für Zeichen	12
7.2	Lesen in ein Zeichen-Array	13
7.3	Vererbung: <code>FileReader</code> und die Basisklasse <code>Reader</code>	14
7.4	Best Practices beim Lesen mit <code>FileReader</code>	14
7.5	Anwendungsfälle für <code>FileReader</code>	14

8	Dateien lesen mit BufferedReader	14
8.1	Zeilenweises Lesen mit readLine()	15
8.2	Puffer und Performance	15
8.3	Lesen bis zu einer bestimmten Anzahl Zeichen	15
8.4	Vererbung und Zusammenspiel mit Reader	16
8.5	Best Practices mit BufferedReader	16
8.6	Typische Anwendungsfälle	16
9	Vergleich: FileReader und BufferedReader	17
9.1	Überblick in Tabellenform	17
9.2	Typischer Einsatz und Kombination	17
9.3	Vergleich wichtiger Methoden	17
9.4	Vererbung und Struktur	18
9.5	Wann welchen Reader verwenden?	18
10	Kombination von Readern und Writern (Best Practices)	18
10.1	Best Practice: Schreiben mit PrintWriter, BufferedWriter und FileWriter	18
10.2	Best Practice: Lesen mit BufferedReader und FileReader	19
10.3	Kombination mit explizitem Zeichensatz (Encoding)	19
11	Parsen von gelesenen Daten	20
11.1	Einfache Werte aus einer Zeile auslesen	20
11.2	Parsen von Wahrheitswerten (boolean)	21
11.3	Best Practices beim Parsen	21
11.4	Anwendungsfälle für Parsen	22
12	Wrapperklassen und optionale Werte beim Parsen	22
12.1	Primitive Typen vs. Wrapperklassen	22
12.2	Parsen mit Wrapperklassen und null	22
12.3	Typische Einsatzgebiete von Wrapperklassen	23
12.4	Weitere sinnvolle Ergänzungen beim Parsen	23
13	Hinweis: try-with-resources und automatisches Schließen	24
13.1	Klassisches try-catch-finally	24
13.2	Moderne Schreibweise: try-with-resources	24
13.3	Vorteile von try-with-resources	25
14	Zusammenfassung: Schreiben, Lesen und Parsen von Dateien in Java	26
14.1	Dateien als Objekte und Streams	26
14.2	Schreiben von Textdateien: FileWriter, BufferedWriter, PrintWriter	26
14.3	Lesen von Textdateien: FileReader und BufferedReader	26
14.4	Kombinationen und Best Practices	27
14.5	Parsen von gelesenen Daten	27
14.6	Gesamtübersicht: Ablauf beim Arbeiten mit Textdateien	28
15	Übungsaufgaben (Einstieg)	29
16	Übungsaufgaben (anspruchsvoller)	30
A	Lösungsskizzen zu den Übungsaufgaben	32
B	Handout: Dateien schreiben, lesen und parsen in Java	39

1 Einführung: Dateien und Streams in Java

In Java wird der Zugriff auf Dateien über das Paket `java.io` realisiert. Wichtige Begriffe:

- **Datei** (`java.io.File`): beschreibt einen Pfad im Dateisystem. Das Anlegen eines **File**-Objekts erzeugt noch keine echte Datei auf der Festplatte.
- **Stream / Writer / Reader**: Klassen, mit denen Daten tatsächlich gelesen oder geschrieben werden. Sie öffnen eine Ressource (z.B. Datei) und müssen am Ende wieder geschlossen werden.

1.1 File-Objekt anlegen

Listing 1: Ein File-Objekt anlegen

```
1 import java.io.File;
2
3 public class DateiAnlegen {
4     public static void main(String[] args) {
5         File f = new File("daten.txt");
6         // Hier wurde auf der Festplatte noch nichts angelegt!
7     }
8 }
```

Wichtige Punkte:

- `new File("daten.txt")` erzeugt nur ein Java-Objekt, das den Pfad beschreibt.
- Die Klasse `File` erbt (wie alle Klassen) von `Object`, ist aber *nicht* die Oberklasse von `FileWriter` oder `PrintWriter`.

1.2 Datei wirklich anlegen mit `createNewFile()`

Listing 2: Datei mit `createNewFile()` anlegen

```
1 import java.io.File;
2 import java.io.IOException;
3
4 public class DateiAnlegenMitCreate {
5     public static void main(String[] args) {
6         File f = new File("daten.txt");
7
8         try {
9             boolean neuAngelegt = f.createNewFile();
10            System.out.println("Datei neu angelegt: " + neuAngelegt);
11        } catch (IOException e) {
12            e.printStackTrace();
13        }
14    }
15 }
```

Rückgabewert und Exception:

- `createNewFile()` liefert einen `boolean`: `true`, wenn die Datei neu erstellt wurde, sonst `false`.
- Die Methode kann eine `IOException` werfen, z.B. bei fehlenden Rechten oder ungültigen Pfaden.

2 Dateien schreiben mit `FileWriter`

Die Klasse `FileWriter` ist ein **Writer** für Zeichen, der direkt in eine Datei schreibt. Sie gehört zum Paket `java.io` und erbt von der abstrakten Basisklasse `Writer`.

2.1 Einfaches Schreiben in eine Datei

Listing 3: Einfaches Schreiben mit FileWriter

```
1 import java.io.FileWriter;
2 import java.io.IOException;
3
4 public class SchreibenMitFileWriter {
5     public static void main(String[] args) {
6         // Datei wird beim Öffnen angelegt (falls sie noch nicht existiert)
7         try (FileWriter fw = new FileWriter("daten.txt")) {
8             fw.write("Erste Zeile\n");
9             fw.write("Zweite Zeile\n");
10        } catch (IOException e) {
11            e.printStackTrace();
12        }
13    }
14 }
```

Wichtige Punkte:

- Der Konstruktor `new FileWriter("daten.txt")` kann eine `IOException` werfen (z.B. kein Schreibrecht).
- Die Methode `write(String s)` hat den Rückgabewert `void`, kann aber ebenfalls eine `IOException` werfen.
- Durch `try-with-resources` wird `fw.close()` am Ende automatisch aufgerufen.

2.2 Datei anhängen statt überschreiben: append

Standardmäßig *überschreibt* `FileWriter` den Inhalt der Datei. Möchte man an eine bestehende Datei *anhängen*, verwendet man den Konstruktor mit zweitem Parameter:

Listing 4: Anhängen mit FileWriter (append)

```
1 import java.io.FileWriter;
2 import java.io.IOException;
3
4 public class AnhängenMitFileWriter {
5     public static void main(String[] args) {
6         // true = an bestehende Datei anhängen (append)
7         try (FileWriter fw = new FileWriter("daten.txt", true)) {
8             fw.write("Neue Zeile am Ende\n");
9         } catch (IOException e) {
10            e.printStackTrace();
11        }
12    }
13 }
```

Rückgabewerte:

- Der Konstruktor hat keinen Rückgabewert (wie alle Konstruktoren), kann aber eine `IOException` auslösen.
- `write(...)` und `append(...)` geben ebenfalls nichts¹ zurück oder einen `Writer` zurück, können aber eine `IOException` werfen.

¹Bei `Writer` selbst liefert `append(...)` ein `Writer`-Objekt zurück, sodass man Methoden verketteten kann. In der Praxis wird dieser Rückgabewert oft ignoriert.

2.3 Die Methoden `close()` und `flush()`

Beim Schreiben in Dateien werden Daten oft zuerst in einen *Puffer* im Speicher geschrieben. Wichtige Methoden sind daher:

- `flush()` – schreibt den Puffer sofort in die Datei, ohne zu schließen.
- `close()` – schreibt (falls nötig) den Rest des Puffers und schließt die Datei. Danach darf der `Writer` nicht mehr verwendet werden.

Listing 5: `flush()` und `close()` verwenden

```
1  import java.io.FileWriter;
2  import java.io.IOException;
3
4  public class FlushUndCloseBeispiel {
5      public static void main(String[] args) {
6          FileWriter fw = null;
7          try {
8              fw = new FileWriter("daten.txt", true); // anhängen
9              fw.write("Zwischenstand...");
10             fw.flush(); // stellt sicher, dass bisherige Daten geschrieben sind
11             fw.write(" noch mehr Text\n");
12         } catch (IOException e) {
13             e.printStackTrace();
14         } finally {
15             if (fw != null) {
16                 try {
17                     fw.close(); // schließt Datei sicher
18                 } catch (IOException e) {
19                     e.printStackTrace();
20                 }
21             }
22         }
23     }
24 }
```

In modernem Code verwendet man meistens `try-with-resources`, dadurch ist ein explizites `close()` im `finally`-Block nicht mehr nötig.

2.4 Vererbung: `FileWriter` und die Basisklasse `Writer`

In der Klasse-Hierarchie gilt (vereinfacht):

- `Writer` ist eine abstrakte Basisklasse für alle Zeichen-Schreiber in `java.io`.
- `FileWriter` erbt von `Writer` und ist auf das Schreiben in Dateien spezialisiert.
- Methoden wie `write()`, `append()`, `flush()` und `close()` stammen aus der Basisklasse `Writer` und werden in `FileWriter` konkret umgesetzt.

3 Dateien schreiben mit `BufferedWriter`

Der `BufferedWriter` arbeitet „vorgelagert“ vor einem anderen `Writer` (z.B. einem `FileWriter`) und sammelt Daten zunächst in einem Puffer im Arbeitsspeicher. Dadurch werden weniger Zugriffe auf die Festplatte nötig, was in der Regel **effizienter** ist.

3.1 Grundidee: Schreiben mit Puffer

Ohne Puffer kann jeder `write(...)`-Aufruf direkt einen Zugriff auf die Datei auslösen. Mit `BufferedWriter` werden viele kleine Schreibzugriffe zu einem größeren Block zusammengefasst.

Listing 6: Schreiben mit `BufferedWriter`

```
1  import java.io.BufferedWriter;
2  import java.io.FileWriter;
3  import java.io.IOException;
4
5  public class SchreibenMitBufferedWriter {
6      public static void main(String[] args) {
7          // FileWriter wird von BufferedWriter eingepackt
8          try (BufferedWriter bw =
9              new BufferedWriter(new FileWriter("daten.txt"))) {
10
11              bw.write("Erste Zeile");
12              bw.newLine(); // plattformabhängiger Zeilenumbruch
13              bw.write("Zweite Zeile");
14          } catch (IOException e) {
15              e.printStackTrace();
16          }
17      }
18  }
```

Wichtige Punkte:

- `BufferedWriter` wird mit einem anderen `Writer` kombiniert, z.B. `new BufferedWriter(new FileWriter(...))`.
- `newLine()` schreibt einen Zeilenumbruch, ohne dass man selbst "" nängen muss.
- Durch `try-with-resources` wird am Ende automatisch `bw.close()` aufgerufen, wodurch auch der `FileWriter` geschlossen wird.

3.2 Anhängen an eine Datei mit `BufferedWriter`

Ob `BufferedWriter` an eine Datei *anhängt* oder sie *überschreibt*, hängt vom inneren `FileWriter` ab. Hier wird der zweite Konstruktor-Parameter auf `true` gesetzt, um an eine bestehende Datei anzuhängen:

Listing 7: Anhängen mit `BufferedWriter`

```
1  import java.io.BufferedWriter;
2  import java.io.FileWriter;
3  import java.io.IOException;
4
5  public class AnhaengenMitBufferedWriter {
6      public static void main(String[] args) {
7          try (BufferedWriter bw =
8              new BufferedWriter(new FileWriter("daten.txt", true))) {
9
10              bw.write("Neue Zeile am Ende");
11              bw.newLine();
12          } catch (IOException e) {
13              e.printStackTrace();
14          }
15      }
16  }
```

Rückgabewerte und Exceptions:

- Die Konstruktoren von `BufferedWriter` und `FileWriter` haben keinen Rückgabewert, können aber eine `IOException` auslösen (z.B. Rechteproblem, ungültiger Pfad).
- Methoden wie `write(...)` und `newLine()` liefern den Rückgabewert `void`, können jedoch ebenfalls eine `IOException` werfen.

3.3 Puffer kontrollieren: `flush()` und `close()`

Da `BufferedWriter` intern puffert, ist das Verhalten von `flush()` und `close()` besonders wichtig:

- `flush()`: schreibt den aktuellen Inhalt des Puffers sofort in die Datei, ohne den Writer zu schließen.
- `close()`: ruft intern noch einmal `flush()` auf (falls nötig) und schließt dann den Writer und die darunterliegende Ressource (z.B. den `FileWriter`).

Listing 8: `flush()` und `close()` mit `BufferedWriter`

```

1  import java.io.BufferedWriter;
2  import java.io.FileWriter;
3  import java.io.IOException;
4
5  public class FlushUndCloseMitBufferedWriter {
6      public static void main(String[] args) {
7          BufferedWriter bw = null;
8          try {
9              bw = new BufferedWriter(new FileWriter("daten.txt", true));
10
11              bw.write("Teil 1...");
12              bw.flush(); // bisheriger Inhalt wird sicher in die Datei
                          geschrieben
13
14              bw.write(" Teil 2\n");
15          } catch (IOException e) {
16              e.printStackTrace();
17          } finally {
18              if (bw != null) {
19                  try {
20                      bw.close(); // schreibt Rest und schließt Datei
21                  } catch (IOException e) {
22                      e.printStackTrace();
23                  }
24              }
25          }
26      }
27  }
```

Hinweis: In modernem Code verwendet man meistens `try-with-resources`, wodurch ein explizites `close()` im `finally`-Block überflüssig wird.

3.4 Vererbung und Zusammenspiel mit `Writer`

In der Klassenhierarchie (vereinfacht) gilt:

- `Writer` ist die abstrakte Basisklasse für alle zeichenbasierten Ausgabeströme in `java.io`.
- `BufferedWriter` erbt von `Writer` und fügt einen Ausgabepuffer hinzu.
- `FileWriter` erbt ebenfalls von `Writer` und kümmert sich konkret um das Schreiben in Dateien.

Das Zusammenspiel sieht also so aus:

BufferedWriter → FileWriter → Datei im Dateisystem

Methoden wie `write()`, `flush()` und `close()` werden in der Basisklasse `Writer` definiert und in den Unterklassen konkret implementiert bzw. erweitert.

4 Dateien schreiben mit `PrintWriter`

Die Klasse `PrintWriter` macht das Schreiben von Text in Dateien besonders bequem. Sie bietet Methoden wie `print`, `println` und `printf`, die Sie bereits von `System.out.println` kennen.

4.1 Grundidee: Komfortable Textausgabe

Im Gegensatz zu `FileWriter` arbeitet `PrintWriter` mit hochleveligen Methoden zur Textausgabe. Statt nur `write` zu benutzen, können Sie zum Beispiel direkt Zahlen, Booleans oder Strings mit `println` ausgeben.

Listing 9: Einfaches Schreiben mit `PrintWriter`

```
1 import java.io.FileWriter;
2 import java.io.IOException;
3 import java.io.PrintWriter;
4
5 public class SchreibenMitPrintWriter {
6     public static void main(String[] args) {
7         try (PrintWriter pw =
8             new PrintWriter(new FileWriter("daten.txt"))) {
9
10            pw.println("Hallo Welt");
11            pw.println(123); // schreibt "123" + Zeilenumbruch
12            pw.println(true); // schreibt "true" + Zeilenumbruch
13
14            pw.printf("Zahl: %d\n", 42); // formatierte Ausgabe
15        } catch (IOException e) {
16            e.printStackTrace();
17        }
18    }
19 }
```

Wichtige Punkte:

- `PrintWriter` wird hier auf einen `FileWriter` aufgesetzt: `new PrintWriter(new FileWriter(...))`.
- Methoden wie `print(...)` und `println(...)` haben den Rückgabewert `void`.
- `printf(...)` erlaubt formatierte Ausgabe ähnlich wie in C (Format-Strings, z.B. `"%d"`, `"%s"`).

4.2 Anhängen an eine Datei mit `PrintWriter`

Ob angehängt oder überschrieben wird, hängt vom darunterliegenden `FileWriter` (oder einem anderen `Writer`) ab. Um an eine bestehende Datei anzuhängen, wird der zweite Konstruktorparameter von `FileWriter` auf `true` gesetzt:

Listing 10: Anhängen mit `PrintWriter`

```
1 import java.io.FileWriter;
2 import java.io.IOException;
3 import java.io.PrintWriter;
4
5 public class AnhaengenMitPrintWriter {
```



```

6 public static void main(String[] args) {
7     try (PrintWriter pw =
8         new PrintWriter(new FileWriter("daten.txt", true))) {
9
10        pw.println("Neue Zeile am Ende");
11    } catch (IOException e) {
12        e.printStackTrace();
13    }
14 }
15 }

```

Rückgabewerte und Exceptions:

- Der Konstruktor von `PrintWriter` hat keinen Rückgabewert.
- Der eingebaute `FileWriter`-Konstruktor kann eine `IOException` auslösen (Pfad/Rechte-Problem), daher das `try-catch`.
- Methoden wie `print`, `println` und `printf` werfen selbst normalerweise keine `IOException`, sondern merken Fehler intern.

4.3 Fehlerbehandlung mit `checkError()`

Im Unterschied zu `FileWriter` oder `BufferedWriter` werden Fehler beim Schreiben mit `PrintWriter` typischerweise *nicht* als `IOException` weitergereicht. Stattdessen können Sie mit `checkError()` prüfen, ob beim Schreiben ein Fehler aufgetreten ist.

Listing 11: Fehlerprüfung mit `checkError()`

```

1 import java.io.FileWriter;
2 import java.io.IOException;
3 import java.io.PrintWriter;
4
5 public class PrintWriterCheckError {
6     public static void main(String[] args) {
7         try (PrintWriter pw =
8             new PrintWriter(new FileWriter("daten.txt"))) {
9
10            pw.println("Testzeile");
11
12            if (pw.checkError()) {
13                System.out.println("Beim Schreiben ist ein Fehler aufgetreten.");
14            }
15        } catch (IOException e) {
16            e.printStackTrace();
17        }
18    }
19 }

```

Rückgabewert von `checkError()`:

- `checkError()` liefert einen `boolean`: `true`, wenn ein Fehler aufgetreten ist, sonst `false`.

4.4 `flush()` und `close()` bei `PrintWriter`

Auch `PrintWriter` besitzt die Methoden `flush()` und `close()`, um den Ausgabepuffer zu kontrollieren:

- `flush()`: schreibt den internen Puffer sofort in die zugrunde liegende Ressource (z.B. Datei), ohne zu schließen.
- `close()`: ruft intern noch einmal `flush()` auf (falls nötig) und schließt anschließend den Writer.

Listing 12: flush() und close() mit PrintWriter

```

1  import java.io.FileWriter;
2  import java.io.IOException;
3  import java.io.PrintWriter;
4
5  public class FlushUndCloseMitPrintWriter {
6      public static void main(String[] args) {
7          PrintWriter pw = null;
8          try {
9              pw = new PrintWriter(new FileWriter("daten.txt", true));
10
11             pw.print("Teil 1...");
12             pw.flush(); // bisherige Daten sicher schreiben
13
14             pw.println(" Teil 2");
15         } catch (IOException e) {
16             e.printStackTrace();
17         } finally {
18             if (pw != null) {
19                 pw.close(); // schließt den Writer und die Datei
20             }
21         }
22     }
23 }

```

In modernerem Code wird meist wieder `try-with-resources` verwendet, sodass ein expliziter `finally`-Block entfällt.

4.5 Vererbung und Zusammenarbeit mit anderen Writer-Klassen

Vereinfacht sieht die Struktur so aus:

- `Writer` ist die abstrakte Basisklasse für zeichenbasierte Ausgaben.
- `FileWriter` erbt von `Writer` und schreibt direkt in Dateien.
- `BufferedWriter` erbt von `Writer` und fügt einen Puffer hinzu.
- `PrintWriter` ist eine eigenständige Klasse, die `Writer` erweitert und Komfortmethoden wie `print`, `println` und `printf` bereitstellt. Er arbeitet typischerweise „obenauf“ auf einem anderen `Writer` oder `OutputStream`.

Typischer Aufbau beim Schreiben in eine Datei mit Puffer und komfortabler Ausgabe:

`PrintWriter` → `BufferedWriter` → `FileWriter` → Datei

So kombiniert man die Vorteile aller Klassen:

- `FileWriter`: Zugang zur Datei.
- `BufferedWriter`: effizientes Puffern.
- `PrintWriter`: bequeme Methoden wie `println` und `printf`.

5 Vergleich: `FileWriter`, `BufferedWriter` und `PrintWriter`

In diesem Abschnitt werden die drei wichtigen Klassen zum Schreiben von Text in Dateien gegenübergestellt: `FileWriter`, `BufferedWriter` und `PrintWriter`.

Klasse	Hauptzweck / Eigenschaften
FileWriter	Schreibt Zeichen direkt in eine Datei. Einfacher, direkter Datei-Writer ohne zusätzlichen Puffer. Bietet nur grundlegende <code>write(...)</code> - und <code>append(...)</code> -Methoden.
BufferedWriter	Schreibt Zeichen gepuffert in eine Datei. Liegt „vor“ einem anderen Writer (z.B. <code>FileWriter</code>) und sammelt Daten im Speicher, bevor sie in größeren Blöcken in die Datei geschrieben werden. Bietet zusätzlich <code>newLine()</code> .
PrintWriter	Komfortable Textausgabe mit Methoden wie <code>print</code> , <code>println</code> und <code>printf</code> . Arbeitet typischerweise auf einem anderen Writer (z.B. <code>FileWriter</code> oder <code>BufferedWriter</code>) oder einem <code>OutputStream</code> . Fehler werden meist intern verwaltet (<code>checkError()</code>).

Tabelle 1: Vergleich von `FileWriter`, `BufferedWriter` und `PrintWriter`

5.1 Überblick in Tabellenform

5.2 Typischer Einsatz und Kombinationen

Oft werden die Klassen nicht allein, sondern in Kombination verwendet. Eine typische Schichtung sieht so aus:

`PrintWriter` → `BufferedWriter` → `FileWriter` → Datei

Damit erhält man gleichzeitig:

- direkten Dateizugriff (`FileWriter`),
- effizientes Puffern (`BufferedWriter`),
- komfortable Ausgabe (`PrintWriter`).

5.3 Vergleich wichtiger Methoden

5.4 Vererbung und Fehlerverhalten

- `Writer` ist die abstrakte Basisklasse für alle zeichenbasierten Ausgabeströme in `java.io`.
- `FileWriter` und `BufferedWriter` erben von `Writer`. Ihre Methoden wie `write()`, `append()`, `flush()` und `close()` können eine `IOException` werfen.
- `PrintWriter` erweitert das Konzept um komfortable Ausgabe-Methoden. Beim Schreiben werden Fehler in der Regel *nicht* als `IOException` geworfen, sondern intern gespeichert. Mit `checkError()` kann geprüft werden, ob ein Fehler aufgetreten ist.

Methode	Wo vorhanden?	Bemerkung
<code>write(...)</code>	<code>FileWriter</code> , <code>BufferedWriter</code> (von <code>Writer</code> geerbt)	Grundlegende Methode zum Schreiben von Zeichen bzw. Strings. Kann eine <code>IOException</code> werfen.
<code>append(...)</code>	<code>FileWriter</code> , <code>BufferedWriter</code> (von <code>Writer</code> geerbt)	Hängt Text an das Ende an. Der Rückgabewert ist ein <code>Writer</code> (Methodenverkettung möglich), wird aber oft ignoriert.
<code>newLine()</code>	<code>BufferedWriter</code>	Fügt einen zeilenabhängigen Zeilenumbruch ein, ohne direkt <code>"\n"</code> zu schreiben.
<code>print(...)</code> / <code>println(...)</code>	<code>PrintWriter</code>	Komfortable Ausgabe für Strings, Zahlen, Booleans usw. Rückgabe ist <code>void</code> , Fehler werden intern gemerkt.
<code>printf(...)</code>	<code>PrintWriter</code>	Formatierte Ausgabe mit Platzhaltern (z.B. <code>"%d"</code> , <code>"%s"</code>). Ähnlich wie in C.
<code>flush()</code>	alle (von <code>Writer</code>)	Schreibt den aktuellen Puffer sofort in die Datei, ohne zu schließen. Wichtig bei gepufferten Schreibern.
<code>close()</code>	alle (von <code>Writer</code>)	Schreibt ggf. den Rest des Puffers und schließt die Ressource. Danach darf der <code>Writer</code> nicht mehr verwendet werden.
<code>checkError()</code>	<code>PrintWriter</code>	Liefert einen <code>boolean</code> (<code>true</code> bei Fehler). Dient zur Fehlerprüfung, da <code>PrintWriter</code> Exceptions meistens nicht direkt weiterreicht.

Tabelle 2: Vergleich wichtiger Methoden der Writer-Klassen

6 Dateien lesen in Java – Einführung

Beim Lesen von Textdateien in Java werden vor allem die Klassen `FileReader` und `BufferedReader` aus dem Paket `java.io` verwendet.

- **`FileReader`**: liest Zeichen direkt aus einer Datei.
- **`BufferedReader`**: liest gepuffert (effizienter) und bietet komfortable Methode `readLine()` zum Lesen ganzer Zeilen.

Beim Lesen von Zeichen spielt die interne Darstellung als **Unicode**-Zeichen eine Rolle. Historisch wird oft noch der Begriff „ASCII“ verwendet: ein Zeichensatz, in dem Buchstaben durch ganze Zahlen (Codes) repräsentiert werden.

In Java wird beim Lesen aus einer Datei der zugrunde liegende Byte-Strom anhand eines Zeichensatzes (Encoding, z.B. UTF-8) in Zeichen (`char`) umgewandelt.

7 Dateien lesen mit `FileReader`

Die Klasse `FileReader` ist ein spezialisierter `Reader` zum Lesen von Zeichen aus Dateien. Sie gehört zum Paket `java.io` und erbt von der abstrakten Basisklasse `Reader`.

7.1 Einfaches Lesen Zeichen für Zeichen

Listing 13: Lesen mit `FileReader` (Zeichen für Zeichen)

```

1  import java.io.FileReader;
2  import java.io.IOException;
3

```

```

4 public class LesenMitFileReader {
5     public static void main(String[] args) {
6         FileReader fr = null;
7         try {
8             fr = new FileReader("daten.txt");
9
10            int code; // Rückgabewert von read() ist int
11            while ((code = fr.read()) != -1) {
12                char c = (char) code; // Umwandlung von int nach char
13                System.out.print(c);
14            }
15        } catch (IOException e) {
16            e.printStackTrace();
17        } finally {
18            if (fr != null) {
19                try {
20                    fr.close();
21                } catch (IOException e) {
22                    e.printStackTrace();
23                }
24            }
25        }
26    }
27 }

```

Rückgabewert und ASCII/Zeichen:

- Die Methode `int read()` liest das *nächste Zeichen* aus der Datei und liefert seinen Code als `int` zurück.
- Bei einem normalen Zeichen ist der Rückgabewert eine nichtnegative ganze Zahl (z.B. 65 für 'A' im ASCII/Unicode).
- Am Dateiende liefert `read()` den Wert -1.
- Durch `(char) code` wird der Ganzzahlcode in ein Java-Zeichen vom Typ `char` umgewandelt.

Exceptions:

- Der Konstruktor `new FileReader("daten.txt")` kann eine `IOException` werfen (z.B. wenn die Datei nicht existiert oder nicht lesbar ist).
- Die Methode `read()` kann ebenfalls eine `IOException` werfen (z.B. bei I/O-Fehlern zur Laufzeit).

7.2 Lesen in ein Zeichen-Array

Statt Zeichen für Zeichen zu lesen, können mehrere Zeichen auf einmal in ein Array eingelesen werden:

Listing 14: Lesen in ein char-Array

```

1 import java.io.FileReader;
2 import java.io.IOException;
3
4 public class LesenInArray {
5     public static void main(String[] args) {
6         try (FileReader fr = new FileReader("daten.txt")) {
7             char[] puffer = new char[1024];
8             int anzahlGelesen;
9
10            while ((anzahlGelesen = fr.read(puffer)) != -1) {
11                // anzahlGelesen gibt an, wie viele Zeichen tatsächlich gelesen
                wurden

```

```

12         String text = new String(puffer, 0, anzahlGelesen);
13         System.out.print(text);
14     }
15     } catch (IOException e) {
16         e.printStackTrace();
17     }
18 }
19 }

```

Rückgabewert:

- `int read(char[] b)` liest bis zu „b.length“ Zeichen und liefert die tatsächliche Anzahl gelesener Zeichen zurück.
- Am Dateiende wird -1 zurückgegeben.

7.3 Vererbung: `FileReader` und die Basisklasse `Reader`

In der Klassenhierarchie (vereinfacht) gilt:

- `Reader` ist die abstrakte Basisklasse für alle zeichenbasierten Eingabeströme in `java.io`.
- `FileReader` erbt von `Reader` und ist auf das Lesen aus Dateien spezialisiert.
- Methoden wie `read()`, `close()` stammen aus der Basisklasse `Reader` und werden in `FileReader` konkret umgesetzt.

7.4 Best Practices beim Lesen mit `FileReader`

- Verwenden Sie nach Möglichkeit **try-with-resources**, damit der `Reader` auch bei Fehlern sicher geschlossen wird.
- Für einfache, kleine Dateien kann `FileReader` direkt verwendet werden.
- Für zeilenweises Lesen und bessere Performance wird in der Praxis meist ein `BufferedReader` „um“ den `FileReader` gelegt.
- Achten Sie bei Textdateien auf das richtige **Encoding** (Zeichensatz). `FileReader` verwendet die Standard- Zeichencodierung des Systems. Für explizite Encodings ist `InputStreamReader` (mit Angabe des Charsets) die flexiblere Wahl.

7.5 Anwendungsfälle für `FileReader`

Typische Einsatzgebiete:

- Einfache Konfigurations- oder Textdateien lesen.
- Dateien Zeichen für Zeichen untersuchen (z.B. für Parser, Zählen bestimmter Zeichen, einfache Verschlüsselungen).
- Basis für einen `BufferedReader`, um effizient zeilenweise zu lesen.

8 Dateien lesen mit `BufferedReader`

Der `BufferedReader` liest Zeichen „gepuffert“ aus einer Quelle (z.B. aus einer Datei) und bietet vor allem die sehr praktische Methode `readLine()` zum Lesen ganzer Zeilen.

8.1 Zeilenweises Lesen mit `readLine()`

Listing 15: Zeilenweises Lesen mit `BufferedReader`

```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.IOException;
4
5 public class LesenMitBufferedReader {
6     public static void main(String[] args) {
7         try (BufferedReader br =
8             new BufferedReader(new FileReader("daten.txt"))) {
9
10             String zeile; // Rueckgabewert von readLine()
11             while ((zeile = br.readLine()) != null) {
12                 System.out.println(zeile);
13             }
14         } catch (IOException e) {
15             e.printStackTrace();
16         }
17     }
18 }
```

Rückgabewert von `readLine()`:

- `readLine()` liefert eine `String`-Referenz zurück, die den Inhalt der nächsten Zeile enthält (ohne Zeilenumbruchzeichen am Ende).
- Am Dateiende gibt `readLine()` den Wert `null` zurück. Dies ist das Abbruchkriterium der Schleife.

Exceptions:

- Der Konstruktor `new FileReader("daten.txt")` kann eine `IOException` werfen (Datei fehlt, keine Rechte usw.).
- Der Konstruktor von `BufferedReader` kann indirekt ebenfalls betroffen sein, wenn der übergebene `Reader` nicht korrekt erstellt werden konnte.
- `readLine()` kann eine `IOException` werfen, z.B. bei I/O-Fehlern während des Lesens.

8.2 Puffer und Performance

Der `BufferedReader` sammelt Daten intern in einem Puffer. Dadurch müssen nicht für jedes einzelne Zeichen oder für jede kleine Leseoperation Zugriffe auf das Dateisystem erfolgen.

- Ohne Puffer können viele kleine Lesezugriffe langsam sein.
- Mit `BufferedReader` werden in der Praxis weniger, dafür grössere Blöcke gelesen → oft deutlich schneller.

8.3 Lesen bis zu einer bestimmten Anzahl Zeichen

Neben `readLine()` kann `BufferedReader` auch direkt in Zeichen-Arrays lesen (die Methoden werden von der Basisklasse `Reader` geerbt):

Listing 16: Gepuffertes Lesen in ein `char`-Array

```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.IOException;
4
```

```

5 public class BufferedLesenInArray {
6     public static void main(String[] args) {
7         try (BufferedReader br =
8             new BufferedReader(new FileReader("daten.txt"))) {
9
10            char[] puffer = new char[1024];
11            int anzahlGelesen;
12
13            while ((anzahlGelesen = br.read(puffer)) != -1) {
14                String text = new String(puffer, 0, anzahlGelesen);
15                System.out.print(text);
16            }
17        } catch (IOException e) {
18            e.printStackTrace();
19        }
20    }
21 }

```

Rückgabewert:

- `int read(char[] b)` liest bis zu „b.length“ Zeichen und liefert die tatsächlich gelesene Anzahl zurück.
- Am Dateiende wird -1 zurückgegeben.

8.4 Vererbung und Zusammenspiel mit Reader

In der Klassenhierarchie (vereinfacht) gilt:

- **Reader** ist die abstrakte Basisklasse für alle zeichenbasierten Eingabeströme.
- **FileReader** erbt von **Reader** und liest Zeichen direkt aus einer Datei.
- **BufferedReader** erbt ebenfalls von **Reader** und fügt einen Lesepuffer sowie die Methode `readLine()` hinzu.

Typischer Aufbau beim Lesen aus einer Datei:

`BufferedReader` → `FileReader` → Datei

8.5 Best Practices mit BufferedReader

- Verwenden Sie **try-with-resources**, damit der **BufferedReader** (und der darunterliegende **FileReader**) automatisch geschlossen wird.
- Nutzen Sie für Textdateien nach Möglichkeit zeilenweises Lesen mit `readLine()`, wenn komplette Zeilen verarbeitet werden sollen.
- Für Dateien mit festem Format oder grossen Textmengen ist **BufferedReader** meist deutlich effizienter und praktischer als ein reiner **FileReader**.
- Wenn ein bestimmtes Encoding wichtig ist (z.B. UTF-8), kann ein **InputStreamReader** mit Charset-Angabe verwendet und anschliessend in einen **BufferedReader** eingepackt werden.

8.6 Typische Anwendungsfälle

- Einlesen von Konfigurationsdateien oder Logdateien zeilenweise.
- Verarbeitung von Textdateien, bei denen jede Zeile eine Bedeutung hat (z.B. CSV-Dateien, einfache Protokolle).
- Basis für weitere Auswertungsschritte: Parsen, Filtern, Suchen in Textdateien usw.

9 Vergleich: FileReader und BufferedReader

In diesem Abschnitt werden die beiden wichtigsten Klassen zum Lesen von Textdateien gegenübergestellt: `FileReader` und `BufferedReader`.

9.1 Überblick in Tabellenform

Klasse	Hauptzweck / Eigenschaften
<code>FileReader</code>	Liest Zeichen direkt aus einer Datei. Einfacher, direkter Datei-Reader ohne zusätzlichen Puffer. Bietet grundlegende <code>read(...)</code> -Methoden zum Lesen von einzelnen Zeichen oder Zeichenblöcken.
<code>BufferedReader</code>	Liest Zeichen gepuffert aus einer Quelle (z.B. <code>FileReader</code>). Fügt einen Lesepuffer hinzu und bietet die komfortable Methode <code>readLine()</code> zum zeilenweisen Lesen. Ist in der Praxis oft effizienter und bequemer.

Tabelle 3: Vergleich von `FileReader` und `BufferedReader`

9.2 Typischer Einsatz und Kombination

Meistens wird `BufferedReader` nicht alleine verwendet, sondern auf einen anderen `Reader` gesetzt, zum Beispiel auf einen `FileReader`:

`BufferedReader` → `FileReader` → Datei

Damit erhält man gleichzeitig:

- direkten Dateizugriff (`FileReader`),
- effizientes Puffern und zeilenweises Lesen (`BufferedReader`).

9.3 Vergleich wichtiger Methoden

Methode	Wo vorhanden?	Bemerkung
<code>int read()</code>	<code>FileReader</code> , <code>BufferedReader</code> (von <code>Reader</code> geerbt)	Liest das nächste Zeichen und liefert dessen Code als <code>int</code> zurück. Rückgabewert <code>-1</code> signalisiert das Dateiende. Kann eine <code>IOException</code> werfen.
<code>int read(char[] b)</code>	<code>FileReader</code> , <code>BufferedReader</code> (von <code>Reader</code> geerbt)	Liest bis zu <code>b.length</code> Zeichen in ein Array. Liefert die tatsächlich gelesene Anzahl oder <code>-1</code> am Dateiende. Kann eine <code>IOException</code> werfen.
<code>String readLine()</code>	nur <code>BufferedReader</code>	Liest eine komplette Zeile (ohne Zeilenumbruch am Ende) als <code>String</code> . Liefert <code>null</code> am Dateiende. Kann eine <code>IOException</code> werfen.
<code>close()</code>	beide (von <code>Reader</code> geerbt)	Schließt den Reader und gibt die Ressource frei. Danach darf der Reader nicht mehr verwendet werden.

Tabelle 4: Vergleich wichtiger Methoden der Reader-Klassen

9.4 Vererbung und Struktur

Vereinfacht sieht die Klassenhierarchie so aus:

- **Reader** ist die abstrakte Basisklasse für alle zeichenbasierten Eingabeströme in `java.io`.
- **FileReader** erbt von **Reader** und liest direkt aus Dateien.
- **BufferedReader** erbt ebenfalls von **Reader** und fügt einen Puffer sowie `readLine()` hinzu.

9.5 Wann welchen Reader verwenden?

- Verwenden Sie **FileReader**, wenn Sie sehr einfache Leseoperationen brauchen oder gezielt Zeichen/Blöcke analysieren möchten.
- Verwenden Sie **BufferedReader**, wenn Sie Textdateien zeilenweise verarbeiten möchten oder Wert auf Performance legen (z.B. bei größeren Dateien).
- In der Praxis: häufig wird direkt `new BufferedReader(new FileReader("datei.txt"))` verwendet, um beide Vorteile zu kombinieren.

10 Kombination von Readern und Writern (Best Practices)

In der Praxis werden die Klassen zum Lesen und Schreiben von Dateien oft „gestapelt“, um die Vorteile mehrerer Klassen zu kombinieren. Typische Kombinationen sind:

- Schreiben: `PrintWriter` → `BufferedWriter` → `FileWriter`
- Lesen: `BufferedReader` → `FileReader`

10.1 Best Practice: Schreiben mit `PrintWriter`, `BufferedWriter` und `FileWriter`

Die folgende Kombination nutzt alle Vorteile beim Schreiben in eine Textdatei:

`PrintWriter` → `BufferedWriter` → `FileWriter` → Datei

Listing 17: Kombiniertes Schreiben in eine Datei

```
1  import java.io.BufferedWriter;
2  import java.io.FileWriter;
3  import java.io.IOException;
4  import java.io.PrintWriter;
5
6  public class KombiniertesSchreiben {
7      public static void main(String[] args) {
8          try (PrintWriter pw = new PrintWriter(
9              new BufferedWriter(
10                 new FileWriter("daten.txt", true) // true = anhaengen
11             ))) {
12
13              pw.println("Neue Zeile mit PrintWriter");
14              pw.printf("Zahl: %d%n", 42);
15          } catch (IOException e) {
16              e.printStackTrace();
17          }
18      }
19  }
```

Vorteile der Kombination:

- **FileWriter**: stellt den direkten Zugriff auf die Datei her, optional mit Anhäng-Modus (zweiter Parameter `true`).
- **BufferedWriter**: sorgt für effizientes Puffern, weniger Festplattenzugriffe.
- **PrintWriter**: bietet komfortable Methoden wie `print`, `println` und `printf`.
- **try-with-resources**: sorgt dafür, dass alle beteiligten Writer am Ende automatisch und sicher geschlossen werden.

10.2 Best Practice: Lesen mit `BufferedReader` und `FileReader`

Beim Lesen von Textdateien wird typischerweise ein `BufferedReader` auf einen `FileReader` gesetzt:

`BufferedReader` → `FileReader` → Datei

Listing 18: Kombiniertes Lesen aus einer Datei

```

1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4
5  public class KombiniertesLesen {
6      public static void main(String[] args) {
7          try (BufferedReader br =
8              new BufferedReader(new FileReader("daten.txt"))) {
9
10             String zeile;
11             while ((zeile = br.readLine()) != null) {
12                 System.out.println(zeile);
13             }
14         } catch (IOException e) {
15             e.printStackTrace();
16         }
17     }
18 }
```

Vorteile der Kombination:

- **FileReader**: liest Zeichen direkt aus der Datei.
- **BufferedReader**: liest effizient in größeren Blöcken und ermöglicht zeilenweises Lesen mit `readLine()`.
- Durch das Muster `while ((zeile = br.readLine()) != null)` wird bis zum Dateiende gelesen (null signalisiert EOF).

10.3 Kombination mit explizitem Zeichensatz (Encoding)

Wenn ein bestimmter Zeichensatz (z.B. UTF-8) wichtig ist, kann statt `FileReader` ein `InputStreamReader` verwendet werden, auf den dann ein `BufferedReader` gesetzt wird:

Listing 19: Lesen mit explizitem Encoding (UTF-8)

```

1  import java.io.BufferedReader;
2  import java.io.IOException;
3  import java.io.InputStreamReader;
4  import java.nio.charset.StandardCharsets;
5  import java.nio.file.Files;
6  import java.nio.file.Paths;
7
8  public class LesenMitEncoding {
9      public static void main(String[] args) {
```

```

10     try (BufferedReader br = new BufferedReader(
11         new InputStreamReader(
12             Files.newInputStream(Paths.get("daten.txt")),
13             StandardCharsets.UTF_8))) {
14
15         String zeile;
16         while ((zeile = br.readLine()) != null) {
17             System.out.println(zeile);
18         }
19     } catch (IOException e) {
20         e.printStackTrace();
21     }
22 }
23 }

```

Hinweis:

- Dieses Muster ist dann sinnvoll, wenn sichergestellt werden soll, dass die Datei mit einem bestimmten Encoding (z.B. UTF-8) gelesen wird, unabhängig von den Standardeinstellungen des Systems.

11 Parsen von gelesenen Daten

Beim Lesen von Textdateien mit `BufferedReader` erhalten wir zunächst nur Zeichen oder Strings (Zeilen). In vielen Fällen müssen diese Texte weiterverarbeitet werden, z.B. zu Zahlen oder Einzelwerten. Diesen Schritt nennt man **Parsen**.

11.1 Einfache Werte aus einer Zeile auslesen

Angenommen, jede Zeile einer Datei hat folgendes Format:

Name;Alter;Note

Beispielzeile:

Anna;17;1.5

Dann kann die Zeile mit `split(";")` in einzelne Teile zerlegt und die Teile in passende Datentypen umgewandelt werden:

Listing 20: Parsen einer Zeile mit Werten

```

1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4
5  public class ParsenBeispiel {
6      public static void main(String[] args) {
7          try (BufferedReader br =
8              new BufferedReader(new FileReader("daten.txt"))) {
9
10             String zeile;
11             while ((zeile = br.readLine()) != null) {
12                 // Zeile in Teile aufspalten
13                 String[] teile = zeile.split(";");
14
15                 // Erwartetes Format: Name;Alter;Note
16                 if (teile.length == 3) {
17                     String name = teile[0];
18                     int alter = Integer.parseInt(teile[1]);

```

```

19         double note = Double.parseDouble(teile[2]);
20
21         System.out.println(
22             "Name: " + name + ", Alter: " + alter +
23             ", Note: " + note);
24     } else {
25         System.out.println("Ungueltige Zeile: " + zeile);
26     }
27 }
28 } catch (IOException e) {
29     e.printStackTrace();
30 } catch (NumberFormatException e) {
31     System.out.println("Fehler beim Umwandeln einer Zahl: "
32         + e.getMessage());
33 }
34 }
35 }

```

Wichtige Punkte beim Parsen:

- `split("; ")` zerlegt den String an jedem Semikolon in ein Array von Strings.
- `Integer.parseInt(...)` wandelt einen String in einen ganzzahligen Wert (`int`) um.
- `Double.parseDouble(...)` wandelt einen String in eine Gleitkommazahl (`double`) um.
- Wenn der Text kein gültiges Zahlenformat hat, wird eine `NumberFormatException` ausgelöst.

11.2 Parsen von Wahrheitswerten (boolean)

Auch logische Werte können als Text gespeichert werden, zum Beispiel `true` oder `false`:

Ben;18;true

Listing 21: Parsen eines boolean-Werts

```

1 String zeile = "Ben;18;true";
2 String[] teile = zeile.split(";");
3
4 String name = teile[0];
5 int alter = Integer.parseInt(teile[1]);
6 boolean aktiv = Boolean.parseBoolean(teile[2]);

```

Hinweis:

- `Boolean.parseBoolean("true")` liefert `true`, alle anderen Strings (z.B. `"false"`, `"äbc"`) liefern `false`.

11.3 Best Practices beim Parsen

- Prüfen Sie die **Anzahl der Teile** nach `split`, bevor Sie darauf zugreifen (z.B. `teile.length == 3`).
- Fangen Sie **`NumberFormatException`** ab, um auf falsche Formate reagieren zu können (z.B. Fehlermeldung, Zeile überspringen).
- Trennen Sie möglichst **Lesen** und **Parsen**:
 - Eine Methode liest die Zeilen (mit `BufferedReader`).
 - Eine andere Methode ist für die Umwandlung der Strings in Fachobjekte zuständig.
- Verwenden Sie aussagekräftige Fehlermeldungen, wenn Zeilen nicht dem erwarteten Format entsprechen.

11.4 Anwendungsfälle für Parsen

Typische Situationen, in denen Parsen notwendig ist:

- Einlesen von Listen (z.B. Schülername, Alter, Note) aus Text- oder CSV-Dateien.
- Auswertung von Logdateien, bei denen jede Zeile mehrere Informationen enthält.
- Konfigurationsdateien, in denen Werte als Text gespeichert sind (z.B. Zahlen, Dateipfade, true/false).

12 Wrapperklassen und optionale Werte beim Parsen

Beim Parsen von Textdateien stößt man oft auf die Frage:

- Was passiert, wenn ein Wert „fehlt“ oder leer ist?
- Wie kann man `null` benutzen, um „kein Wert“ zu kennzeichnen?

Für solche Fälle sind die **Wrapperklassen** (z.B. `Integer`, `Double`, `Boolean`) praktisch. Sie sind Objektvarianten der primitiven Typen `int`, `double`, `boolean` usw. und können den Wert `null` annehmen.

12.1 Primitive Typen vs. Wrapperklassen

Primitive	Wrapperklasse	Kann null sein?
<code>int</code>	<code>Integer</code>	ja
<code>double</code>	<code>Double</code>	ja
<code>boolean</code>	<code>Boolean</code>	ja
<code>char</code>	<code>Character</code>	ja

Tabelle 5: Primitive Datentypen und passende Wrapperklassen

Beispiel:

- Ein Feld `int alter`; muss immer eine Zahl enthalten.
- Ein Feld `Integer alter`; kann auch `null` sein (z.B. „Alter unbekannt“).

12.2 Parsen mit Wrapperklassen und null

Angenommen, jede Zeile hat das Format:

```
Name;Alter;Note
```

Das Alter oder die Note dürfen aber auch fehlen, z.B.:

```
Anna;17;1.5
Ben;;2.3      (kein Alter eingetragen)
Chris;18;     (Note fehlt)
```

Mit Wrapperklassen können wir fehlende Werte als `null` kennzeichnen:

Listing 22: Parsen mit Wrapperklassen und null

```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.IOException;
4
```

```

5 public class ParsenMitWrapper {
6     public static void main(String[] args) {
7         try (BufferedReader br =
8             new BufferedReader(new FileReader("daten.txt"))) {
9
10            String zeile;
11            while ((zeile = br.readLine()) != null) {
12                String[] teile = zeile.split(";");
13                if (teile.length != 3) {
14                    System.out.println("Ungueltige Zeile: " + zeile);
15                    continue; // zur nächsten Zeile springen
16                }
17
18                String name = teile[0];
19
20                Integer alter = null;
21                if (!teile[1].isEmpty()) {
22                    alter = Integer.valueOf(teile[1]);
23                }
24
25                Double note = null;
26                if (!teile[2].isEmpty()) {
27                    note = Double.valueOf(teile[2]);
28                }
29
30                System.out.println("Name: " + name
31                    + ", Alter: " + alter
32                    + ", Note: " + note);
33            }
34        } catch (IOException e) {
35            e.printStackTrace();
36        } catch (NumberFormatException e) {
37            System.out.println("Fehler beim Umwandeln einer Zahl: "
38                + e.getMessage());
39        }
40    }
41 }

```

Wichtige Punkte:

- `Integer alter = null;` kennzeichnet: „Alter ist (noch) unbekannt“.
- `!teile[1].isEmpty()` prüft, ob zwischen den Semikolons wirklich etwas steht.
- `Integer.valueOf(...)` und `Double.valueOf(...)` liefern ein Wrapperobjekt (statt eines primitiven Werts).

12.3 Typische Einsatzgebiete von Wrapperklassen

Wrapperklassen sind besonders nützlich, wenn

- ein Wert *optional* ist (z.B. Alter/Note nicht bekannt),
- Sie mit Sammlungen (`ArrayList<Integer>`) arbeiten,
- Sie „kein Wert vorhanden“ mit `null` darstellen wollen.

12.4 Weitere sinnvolle Ergänzungen beim Parsen

Zusammen mit Wrapperklassen ergeben sich häufig diese **Best Practices**:

- Die gelesenen Daten in einer eigenen Klasse speichern, z.B.:

```

1      class Schueler {
2          String name;
3          Integer alter; // darf null sein
4          Double note; // darf null sein
5      }

```

- Eine Methode, die aus einer Zeile ein **Schueler**-Objekt baut (Lesen/Parsen trennen).
- Klare Regeln, was bei Fehlern passiert:
 - Zeile überspringen,
 - Standardwert setzen,
 - oder Programm mit Fehlermeldung abbrechen.

13 Hinweis: try-with-resources und automatisches Schließen

Seit Java 7 gibt es das Konstrukt **try-with-resources**. Es vereinfacht den Umgang mit Ressourcen wie Dateien, weil Reader und Writer automatisch geschlossen werden, ohne dass ein eigener **finally**-Block nötig ist.

13.1 Klassisches try-catch-finally

Früher wurde oft folgender Aufbau verwendet:

Listing 23: Klassisches try-catch-finally

```

1  import java.io.FileWriter;
2  import java.io.IOException;
3
4  public class Klassisch {
5      public static void main(String[] args) {
6          FileWriter fw = null;
7          try {
8              fw = new FileWriter("daten.txt");
9              fw.write("Text...");
10         } catch (IOException e) {
11             e.printStackTrace();
12         } finally {
13             if (fw != null) {
14                 try {
15                     fw.close();
16                 } catch (IOException e) {
17                     e.printStackTrace();
18                 }
19             }
20         }
21     }
22 }

```

Hier muss im **finally**-Block geprüft werden, ob das Objekt nicht **null** ist, und **close()** muss selbst aufgerufen werden.

13.2 Moderne Schreibweise: try-with-resources

Mit **try-with-resources** kann der Code deutlich kürzer und sicherer werden. Alle Ressourcen, die im runden Klammerpaar hinter **try** angelegt werden und das Interface **AutoCloseable** implementieren (z.B. **FileWriter**, **BufferedWriter**, **PrintWriter**, **FileReader**, **BufferedReader**), werden am Ende des Blocks automatisch geschlossen.

Listing 24: try-with-resources beim Schreiben

```

1  import java.io.FileWriter;
2  import java.io.IOException;
3
4  public class Modern {
5      public static void main(String[] args) {
6          try (FileWriter fw = new FileWriter("daten.txt")) {
7              fw.write("Text...\n");
8          } catch (IOException e) {
9              e.printStackTrace();
10         }
11         // Hier ist fw bereits automatisch geschlossen.
12     }
13 }

```

Genauso beim Lesen:

Listing 25: try-with-resources beim Lesen

```

1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4
5  public class ModernLesen {
6      public static void main(String[] args) {
7          try (BufferedReader br =
8              new BufferedReader(new FileReader("daten.txt"))) {
9
10             String zeile;
11             while ((zeile = br.readLine()) != null) {
12                 System.out.println(zeile);
13             }
14         } catch (IOException e) {
15             e.printStackTrace();
16         }
17         // br (und der darunterliegende FileReader) sind jetzt geschlossen.
18     }
19 }

```

13.3 Vorteile von try-with-resources

- Weniger Code: kein eigener finally-Block nötig.
- Weniger Fehlerquellen: Ressourcen werden auch bei Exceptions zuverlässig geschlossen.
- Gut lesbar: direkt sichtbar, welche Ressourcen in diesem Block verwaltet werden.

In dieser Lerndatei sollten nach Möglichkeit alle neuen Beispiele mit try-with-resources geschrieben werden. Das klassische try-catch-finally-Muster ist trotzdem wichtig zu kennen, um älteren Code zu verstehen.

14 Zusammenfassung: Schreiben, Lesen und Parsen von Dateien in Java

In diesem Abschnitt werden die wichtigsten Konzepte aus den vorherigen Kapitel kurz zusammengefasst. Ziel ist ein „roter Faden“, wie man in Java mit Textdateien arbeitet.

14.1 Dateien als Objekte und Streams

- Die Klasse **File** beschreibt einen Pfad im Dateisystem (z.B. "daten.txt"). Ein **File**-Objekt allein liest oder schreibt noch nichts.
- Erst durch **Reader** und **Writer** (z.B. **FileReader**, **FileWriter**) findet der eigentliche Datenzugriff statt.
- Reader sind für das **Lesen** von Zeichen zuständig, Writer für das **Schreiben** von Zeichen.

14.2 Schreiben von Textdateien: **FileWriter**, **BufferedWriter**, **PrintWriter**

FileWriter:

- Schreibt Zeichen direkt in eine Datei.
- Konstruktoren können eine **IOException** werfen.
- **write(...)** und **append(...)** haben den Rückgabewert **void** (bzw. einen **Writer**) und können eine **IOException** werfen.
- Mit dem zweiten Parameter **true** im Konstruktor (**new FileWriter("daten.txt", true)**) wird an die Datei **angehängt** statt sie zu überschreiben.
- **flush()** schreibt den Puffer in die Datei, **close()** schreibt ggf. den Rest und schließt die Datei.

BufferedWriter:

- Erbt von **Writer** und fügt einen Ausgabepuffer hinzu.
- Wird typischerweise „um“ einen **FileWriter** gelegt: **new BufferedWriter(new FileWriter(...))**.
- Reduziert die Anzahl der physischen Schreibzugriffe und verbessert so die Performance.
- Bietet zusätzlich **newLine()** für Zeilenumbrüche.

PrintWriter:

- Bietet komfortable Methoden wie **print**, **println** und **printf** (ähnlich wie **System.out**).
- Arbeitet meist auf einem anderen **Writer** (z.B. **FileWriter** oder **BufferedWriter**).
- Fehler beim Schreiben werden i.d.R. *nicht* als **IOException** weitergereicht, sondern intern gespeichert. Mit **checkError()** (Rückgabewert **boolean**) kann geprüft werden, ob ein Fehler aufgetreten ist.
- Auch hier sind **flush()** und **close()** wichtig, um alle Daten sicher zu schreiben.

14.3 Lesen von Textdateien: **FileReader** und **BufferedReader**

FileReader:

- Erbt von **Reader** und liest Zeichen direkt aus einer Datei.
- **int read()** liest das nächste Zeichen als Ganzzahlcode (z.B. ASCII/Unicode-Code). Am Dateiende liefert **read()** den Wert **-1**.

- `int read(char[] puffer)` liest mehrere Zeichen auf einmal in ein Array und gibt die tatsächliche Anzahl gelesener Zeichen zurück (oder `-1` am Dateiende).
- Methoden wie `read(...)` und `close()` können eine `IOException` werfen.

BufferedReader:

- Erbt ebenfalls von `Reader` und liest gepuffert.
- Wird typischerweise um einen `FileReader` gelegt: `new BufferedReader(new FileReader("daten.txt"))`.
- Bietet die sehr praktische Methode `readLine()` mit dem Rückgabewert `String`.
 - Liefert die nächste Zeile ohne Zeilenumbruchzeichen.
 - Liefert `null` am Dateiende.
- Ist für Textdateien mit zeilenweiser Struktur (z.B. Log- oder CSV-Dateien) besonders gut geeignet.

14.4 Kombinationen und Best Practices

- Schreiben (typische Kombination):

`PrintWriter → BufferedWriter → FileWriter → Datei`

Dadurch werden direkter Dateizugriff, Puffern und bequeme Ausgabe verbunden.

- Lesen (typische Kombination):

`BufferedReader → FileReader → Datei`

So werden effizienter Zugriff und zeilenweises Lesen kombiniert.

- **try-with-resources** sollte wann immer möglich verwendet werden, damit `Reader/Writer` automatisch und sicher geschlossen werden.
- Bei wichtigem Zeichensatz (z.B. UTF-8) ist eine Kombination mit `InputStreamReader` sinnvoll, um das Encoding explizit zu steuern.

14.5 Parsen von gelesenen Daten

- Beim Lesen mit `BufferedReader` (`readLine()`) entstehen zunächst `Strings` (Zeilen).
- Durch Methoden wie `split(";")`, `Integer.parseInt`, `Double.parseDouble` oder `Boolean.parseBoolean` werden diese `Strings` in einzelne Werte umgewandelt.
- Fehlt ein Wert oder ist er leer, bieten sich Wrapperklassen (z.B. `Integer`, `Double`, `Boolean`) an, da sie den Zustand `null` („kein Wert“) erlauben.
- Typische Fehler wie `NumberFormatException` sollten abgefangen werden, um auf ungültige Zeilen reagieren zu können.

14.6 Gesamtübersicht: Ablauf beim Arbeiten mit Textdateien

Ein typischer Ablauf beim Umgang mit Textdateien in Java sieht so aus:

1. **Datei/Bedeutung klären:** Welche Daten stehen in welcher Struktur in der Datei (z.B. eine Zeile pro Datensatz, Trennzeichen usw.)?
2. **Lesen oder Schreiben wählen:**
 - Schreiben: Kombination aus `FileWriter`, `BufferedWriter`, `PrintWriter`.
 - Lesen: Kombination aus `FileReader`, `BufferedReader`.
3. **Daten verarbeiten:** Gelesene Strings parsen, umwandeln und in sinnvollen Strukturen (z.B. Objektklassen wie `Schueler`) speichern.
4. **Fehlerbehandlung:** Mit `IOException` und `NumberFormatException` umgehen, unvollständige oder fehlerhafte Zeilen sinnvoll behandeln.
5. **Ressourcen schliessen:** Mit `try-with-resources` sicherstellen, dass alle Dateien nach Gebrauch wieder korrekt geschlossen werden.

Diese Bausteine bilden zusammen die Grundlage für den praktischen Umgang mit Textdateien in Java.

15 Übungsaufgaben (Einstieg)

In diesem Abschnitt übst du die Grundlagen zum Schreiben, Lesen und einfachen Parsen von Textdateien in Java.

Aufgabe 1: Einfache Textdatei schreiben

Schreibe ein Java-Programm, das eine Datei mit dem Namen `ausgabe.txt` erzeugt und folgende Zeilen hineinschreibt:

```
Hallo Java-Datei!  
Dies ist die zweite Zeile.
```

- Verwende `FileWriter` (ohne Anhängen).
- Nutze `try-with-resources`, damit die Datei am Ende automatisch geschlossen wird.

Aufgabe 2: An eine Datei anhängen

Erweitere dein Programm aus Aufgabe 1 oder schreibe ein neues Programm, das an die Datei `ausgabe.txt` eine weitere Zeile anhängt:

```
Dies ist eine angehängte Zeile.
```

- Verwende den Konstruktor von `FileWriter` mit dem zweiten Parameter `true`, um die Datei im *Anhänge-Modus* zu öffnen.
- Überprüfe das Ergebnis, indem du die Datei im Editor öffnest.

Aufgabe 3: Datei zeilenweise lesen

Schreibe ein Java-Programm, das die Datei `ausgabe.txt` zeilenweise einliest und jede Zeile auf der Konsole ausgibt.

- Verwende `BufferedReader` in Kombination mit `FileReader`.
- Nutze die Methode `readLine()` und eine `while`-Schleife.
- Denke an die Abbruchbedingung mit `null` am Dateende.

Aufgabe 4: Zeichen zählen

Schreibe ein Programm, das eine Textdatei (`text.txt`) Zeichen für Zeichen einliest und die Anzahl aller gelesenen Zeichen auf der Konsole ausgibt.

- Verwende `FileReader` und die Methode `int read()`.
- Lies so lange, bis `read()` den Wert `-1` zurückliefert.
- Zähle, wie viele Zeichen insgesamt gelesen wurden.

Aufgabe 5: Einfache Werte aus einer Zeile parsen

Gegeben ist eine Datei `person.txt` mit genau einer Zeile im folgenden Format:

`Name;Alter;Note`

Beispielinhalt:

`Anna;17;1.5`

Schreibe ein Programm, das

- die Zeile aus der Datei liest,
- die Zeile mit `split(";")` in `Name`, `Alter` und `Note` zerlegt,
- `Alter` in einen `int` und `Note` in ein `double` umwandelt (`parseInt`, `parseDouble`),
- alle Werte formatiert auf der Konsole ausgibt, z.B.:

`Name: Anna, Alter: 17, Note: 1.5`

16 Übungsaufgaben (anspruchsvoller)

In diesem Abschnitt wendest du Schreiben, Lesen und Parsen kombiniert an und arbeitest mit eigenen Klassen und Wrapperklassen.

Aufgabe 6: Liste von Personen einlesen

Es gibt eine Datei `personen.txt`, in der jede Zeile eine Person beschreibt:

`Name;Alter;Note`

Beispiele:

`Anna;17;1.5`

`Ben;18;2.3`

`Chris;16;3.0`

1. Definiere eine Klasse `Schueler` mit den Feldern `name (String)`, `alter (int)` und `note (double)`.
2. Schreibe ein Programm, das alle Zeilen aus `personen.txt` einliest, parst und für jede Zeile ein `Schueler`-Objekt erzeugt.
3. Speichere alle Objekte in einer `ArrayList<Schueler>`.
4. Gib anschließend alle Schüler formatiert auf der Konsole aus.

Aufgabe 7: Optionale Werte mit Wrapperklassen

Erweitere Aufgabe 6: Jetzt dürfen `Alter` und `Note` optional sein. Beispiele für Zeilen in `personen.txt`:

`Anna;17;1.5`

`Ben;;2.3` (kein `Alter` eingetragen)

`Chris;18;` (`Note` fehlt)

1. Ändere die Klasse `Schueler` so, dass `alter` den Typ `Integer` und `note` den Typ `Double` hat.
2. Beim Parsen:
 - Wenn das Feld leer ist (`()`), soll der Wert `null` gesetzt werden.
 - Wenn ein Zahlenfeld nicht leer ist, wandle es mit `Integer.valueOf(...)` bzw. `Double.valueOf(...)` um.
3. Gib bei der Ausgabe klar an, wenn `Alter` oder `Note` nicht vorhanden sind (z.B. „Alter unbekannt“).

Aufgabe 8: Datei filtern und in neue Datei schreiben

Auf Basis der Daten aus Aufgabe 6 oder 7:

1. Lies alle Schüler aus `personen.txt` ein und speichere sie in einer `ArrayList<Schueler>`.
2. Filtere alle Schüler mit einer Note besser als 2.0 (z.B. `note < 2.0`).
3. Schreibe nur diese „guten“ Schüler in eine neue Datei `beste_guten.txt` im gleichen Format:
`Name;Alter;Note`
4. Verwende zum Schreiben eine Kombination aus `PrintWriter`, `BufferedWriter` und `FileWriter`.

Aufgabe 9: Statistik aus Datei berechnen

Gegeben ist wieder eine Datei `personen.txt` im bekannten Format. Schreibe ein Programm, das

1. alle Schüler einliest und parst (wie in Aufgabe 6),
2. die durchschnittliche Note aller Schüler berechnet,
3. die beste und die schlechteste Note bestimmt,
4. das Ergebnis übersichtlich auf der Konsole ausgibt.

Hinweise:

- Ignoriere ggf. Schüler, bei denen die Note fehlt (z.B. `note == null`).
- Überlege dir sinnvolle Variablen für Summe, Anzahl, Minimum und Maximum.

Aufgabe 10: Fehlerrobustes Parsen

In der Datei `personen.txt` können auch fehlerhafte Zeilen vorkommen, z.B.:

```
Anna;17;1.5
Ben;abc;2.3    (abc ist kein gültiges Alter)
Chris;18;
;;    (fast alles fehlt)
```

Schreibe ein Programm, das

1. jede Zeile einliest und versucht zu parsen,
2. bei Format- oder Zahlenfehlern (`NumberFormatException`) die fehlerhafte Zeile meldet,
3. die fehlerhafte Zeile überspringt und mit der nächsten Zeile weitermacht, statt das ganze Programm abzubrechen.

A Lösungsskizzen zu den Übungsaufgaben

Hinweis: Dies sind Lösungsskizzen. Andere Lösungen können ebenfalls korrekt sein, solange sie die Anforderungen erfüllen.

Lösung zu Aufgabe 1: Einfache Textdatei schreiben

```
1  import java.io.FileWriter;
2  import java.io.IOException;
3
4  public class Aufgabe1 {
5      public static void main(String[] args) {
6          try (FileWriter fw = new FileWriter("ausgabe.txt")) {
7              fw.write("Hallo Java-Datei!\n");
8              fw.write("Dies ist die zweite Zeile.\n");
9          } catch (IOException e) {
10             e.printStackTrace();
11         }
12     }
13 }
```

Lösung zu Aufgabe 2: An eine Datei anhängen

```
1  import java.io.FileWriter;
2  import java.io.IOException;
3
4  public class Aufgabe2 {
5      public static void main(String[] args) {
6          try (FileWriter fw = new FileWriter("ausgabe.txt", true)) {
7              fw.write("Dies ist eine angehängte Zeile.\n");
8          } catch (IOException e) {
9              e.printStackTrace();
10         }
11     }
12 }
```

Lösung zu Aufgabe 3: Datei zeilenweise lesen

```
1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4
5  public class Aufgabe3 {
6      public static void main(String[] args) {
7          try (BufferedReader br =
8              new BufferedReader(new FileReader("ausgabe.txt"))) {
9
10             String zeile;
11             while ((zeile = br.readLine()) != null) {
12                 System.out.println(zeile);
13             }
14         } catch (IOException e) {
15             e.printStackTrace();
16         }
17     }
18 }
```


Lösung zu Aufgabe 4: Zeichen zählen

```
1  import java.io.FileReader;
2  import java.io.IOException;
3
4  public class Aufgabe4 {
5      public static void main(String[] args) {
6          int zaehler = 0;
7
8          try (FileReader fr = new FileReader("text.txt")) {
9              int code;
10             while ((code = fr.read()) != -1) {
11                 zaehler++;
12             }
13             System.out.println("Anzahl Zeichen: " + zaehler);
14         } catch (IOException e) {
15             e.printStackTrace();
16         }
17     }
18 }
```

Lösung zu Aufgabe 5: Einfache Werte aus einer Zeile parsen

```
1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4
5  public class Aufgabe5 {
6      public static void main(String[] args) {
7          try (BufferedReader br =
8              new BufferedReader(new FileReader("person.txt"))) {
9
10             String zeile = br.readLine();
11             if (zeile != null) {
12                 String[] teile = zeile.split(";");
13                 String name = teile[0];
14                 int alter = Integer.parseInt(teile[1]);
15                 double note = Double.parseDouble(teile[2]);
16
17                 System.out.println("Name: " + name +
18                                     ", Alter: " + alter +
19                                     ", Note: " + note);
20             }
21         } catch (IOException e) {
22             e.printStackTrace();
23         } catch (NumberFormatException e) {
24             System.out.println("Fehler beim Umwandeln einer Zahl: "
25                                 + e.getMessage());
26         }
27     }
28 }
```

Lösung zu Aufgabe 6: Liste von Personen einlesen

```
1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4  import java.util.ArrayList;
```

```

5  import java.util.List;
6
7  class Schueler {
8      String name;
9      int alter;
10     double note;
11
12     Schueler(String name, int alter, double note) {
13         this.name = name;
14         this.alter = alter;
15         this.note = note;
16     }
17 }
18
19 public class Aufgabe6 {
20     public static void main(String[] args) {
21         List<Schueler> liste = new ArrayList<>();
22
23         try (BufferedReader br =
24             new BufferedReader(new FileReader("personen.txt"))) {
25
26             String zeile;
27             while ((zeile = br.readLine()) != null) {
28                 String[] teile = zeile.split(";");
29                 if (teile.length != 3) continue;
30
31                 String name = teile[0];
32                 int alter = Integer.parseInt(teile[1]);
33                 double note = Double.parseDouble(teile[2]);
34
35                 liste.add(new Schueler(name, alter, note));
36             }
37         } catch (IOException | NumberFormatException e) {
38             e.printStackTrace();
39         }
40
41         for (Schueler s : liste) {
42             System.out.println(s.name + " (" + s.alter + ") - Note: " + s.note);
43         }
44     }
45 }

```

Lösung zu Aufgabe 7: Optionale Werte mit Wrapperklassen

```

1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4  import java.util.ArrayList;
5  import java.util.List;
6
7  class SchuelerOpt {
8      String name;
9      Integer alter; // darf null sein
10     Double note; // darf null sein
11
12     SchuelerOpt(String name, Integer alter, Double note) {
13         this.name = name;
14         this.alter = alter;
15         this.note = note;

```

```

16     }
17 }
18
19 public class Aufgabe7 {
20     public static void main(String[] args) {
21         List<SchuelerOpt> liste = new ArrayList<>();
22
23         try (BufferedReader br =
24             new BufferedReader(new FileReader("personen.txt"))) {
25
26             String zeile;
27             while ((zeile = br.readLine()) != null) {
28                 String[] teile = zeile.split(";");
29                 if (teile.length != 3) continue;
30
31                 String name = teile[0];
32
33                 Integer alter = null;
34                 if (!teile[1].isEmpty()) {
35                     alter = Integer.valueOf(teile[1]);
36                 }
37
38                 Double note = null;
39                 if (!teile[2].isEmpty()) {
40                     note = Double.valueOf(teile[2]);
41                 }
42
43                 liste.add(new SchuelerOpt(name, alter, note));
44             }
45         } catch (IOException | NumberFormatException e) {
46             e.printStackTrace();
47         }
48
49         for (SchuelerOpt s : liste) {
50             String alterText = (s.alter == null) ? "Alter unbekannt" : s.alter.
51                 toString();
52             String noteText = (s.note == null) ? "Note fehlt" : s.note.toString
53                 ();
54             System.out.println(s.name + ": " + alterText + ", " + noteText);
55         }
56     }
57 }

```

Lösung zu Aufgabe 8: Datei filtern und in neue Datei schreiben

```

1  import java.io.BufferedReader;
2  import java.io.BufferedWriter;
3  import java.io.FileReader;
4  import java.io.FileWriter;
5  import java.io.IOException;
6  import java.io.PrintWriter;
7  import java.util.ArrayList;
8  import java.util.List;
9
10 class Schueler2 {
11     String name;
12     int alter;
13     double note;
14

```

```

15     Schueler2(String name, int alter, double note) {
16         this.name = name;
17         this.alter = alter;
18         this.note = note;
19     }
20 }
21
22 public class Aufgabe8 {
23     public static void main(String[] args) {
24         List<Schueler2> liste = new ArrayList<>();
25
26         // Einlesen
27         try (BufferedReader br =
28             new BufferedReader(new FileReader("personen.txt"))) {
29
30             String zeile;
31             while ((zeile = br.readLine()) != null) {
32                 String[] teile = zeile.split(";");
33                 if (teile.length != 3) continue;
34
35                 String name = teile[0];
36                 int alter = Integer.parseInt(teile[1]);
37                 double note = Double.parseDouble(teile[2]);
38
39                 liste.add(new Schueler2(name, alter, note));
40             }
41         } catch (IOException | NumberFormatException e) {
42             e.printStackTrace();
43         }
44
45         // Filtern und schreiben
46         try (PrintWriter pw = new PrintWriter(
47             new BufferedWriter(
48                 new FileWriter("beste_schueler.txt")
49             ))) {
50
51             for (Schueler2 s : liste) {
52                 if (s.note < 2.0) {
53                     pw.println(s.name + ";" + s.alter + ";" + s.note);
54                 }
55             }
56         } catch (IOException e) {
57             e.printStackTrace();
58         }
59     }
60 }

```

Lösung zu Aufgabe 9: Statistik aus Datei berechnen

```

1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4
5  public class Aufgabe9 {
6      public static void main(String[] args) {
7          double summe = 0.0;
8          int anzahl = 0;
9          Double min = null;
10         Double max = null;

```

```

11
12     try (BufferedReader br =
13         new BufferedReader(new FileReader("personen.txt"))) {
14
15         String zeile;
16         while ((zeile = br.readLine()) != null) {
17             String[] teile = zeile.split(";");
18             if (teile.length != 3 || teile[2].isEmpty()) continue;
19
20             try {
21                 double note = Double.parseDouble(teile[2]);
22                 summe += note;
23                 anzahl++;
24
25                 if (min == null || note < min) {
26                     min = note;
27                 }
28                 if (max == null || note > max) {
29                     max = note;
30                 }
31             } catch (NumberFormatException e) {
32                 // fehlerhafte Note ignorieren
33             }
34         }
35     } catch (IOException e) {
36         e.printStackTrace();
37     }
38
39     if (anzahl > 0) {
40         double durchschnitt = summe / anzahl;
41         System.out.println("Durchschnittsnote: " + durchschnitt);
42         System.out.println("Beste Note: " + min);
43         System.out.println("Schlechteste Note: " + max);
44     } else {
45         System.out.println("Keine gültigen Noten gefunden.");
46     }
47 }
48

```

Lösung zu Aufgabe 10: Fehlerrobustes Parsen

```

1  import java.io.BufferedReader;
2  import java.io.FileReader;
3  import java.io.IOException;
4
5  public class Aufgabe10 {
6      public static void main(String[] args) {
7          try (BufferedReader br =
8              new BufferedReader(new FileReader("personen.txt"))) {
9
10             String zeile;
11             while ((zeile = br.readLine()) != null) {
12                 String[] teile = zeile.split(";");
13                 if (teile.length != 3) {
14                     System.out.println("Ungültige Zeile (Anzahl Felder): " + zeile);
15                     ;
16                     continue;
17                 }

```

```

18     try {
19         String name = teile[0];
20         String alterText = teile[1];
21         String noteText = teile[2];
22
23         Integer alter = null;
24         if (!alterText.isEmpty()) {
25             alter = Integer.valueOf(alterText);
26         }
27
28         Double note = null;
29         if (!noteText.isEmpty()) {
30             note = Double.valueOf(noteText);
31         }
32
33         System.out.println("OK: " + name + ", " + alter + ", " + note);
34     } catch (NumberFormatException e) {
35         System.out.println("Fehlerhafte Zahlenangabe in Zeile: " + zeile
36             );
37     }
38 } catch (IOException e) {
39     e.printStackTrace();
40 }
41 }
42 }

```

B Handout: Dateien schreiben, lesen und parsen in Java

1. Wichtige Klassen (Überblick)

Schreiben (Writer):

- `FileWriter`: schreibt Zeichen direkt in Dateien.
- `BufferedWriter`: schreibt gepuffert (schneller), bietet `newLine()`.
- `PrintWriter`: bequeme Methoden `print`, `println`, `printf`.

Lesen (Reader):

- `FileReader`: liest Zeichen direkt aus Dateien.
- `BufferedReader`: liest gepuffert, bietet `readLine()`.

2. Typische Kombinationen

Schreiben:

`PrintWriter` → `BufferedWriter` → `FileWriter` → Datei

Lesen:

`BufferedReader` → `FileReader` → Datei

3. Schreiben mit try-with-resources

```
1 try (PrintWriter pw = new PrintWriter(  
2 new BufferedWriter(  
3 new FileWriter("daten.txt", true)))) { // true = anhaengen  
4  
5     pw.println("Hallo Welt");  
6     pw.printf("Zahl: %d%n", 42);  
7 } catch (IOException e) {  
8     e.printStackTrace();  
9 }  
10 // pw wird automatisch geschlossen
```

Wichtig:

- `try (...)`: alles in den Klammern wird am Blockende automatisch geschlossen.
- `FileWriter("datei", true)`: an Datei anhängen statt überschreiben.

4. Lesen (zeilenweise) mit BufferedReader

```
1 try (BufferedReader br =  
2 new BufferedReader(new FileReader("daten.txt"))) {  
3  
4     String zeile;  
5     while ((zeile = br.readLine()) != null) {  
6         System.out.println(zeile);  
7     }  
8 } catch (IOException e) {  
9     e.printStackTrace();  
10 }
```

Merken:

- `readLine()` → nächste Zeile als String, null am Dateiende.

5. Parsen einer Zeile

Format einer Zeile:

Name;Alter;Note

```
1 String zeile = "Anna;17;1.5";
2 String[] teile = zeile.split(";");
3
4 String name = teile[0];
5 int alter = Integer.parseInt(teile[1]);
6 double note = Double.parseDouble(teile[2]);
```

Merken:

- `split(";")`: String an Trennzeichen aufteilen.
- `parseInt`, `parseDouble`: `String` → `Zahl` (kann `NumberFormatException` werfen).

6. Wrapperklassen für optionale Werte

Beispielzeilen (Alter/Note dürfen fehlen):

Anna;17;1.5

Ben;;2.3 (kein Alter)

Chris;18; (keine Note)

```
1 String[] teile = zeile.split(";");
2
3 String name = teile[0];
4 Integer alter = null;
5 if (!teile[1].isEmpty()) {
6     alter = Integer.valueOf(teile[1]);
7 }
8
9 Double note = null;
10 if (!teile[2].isEmpty()) {
11     note = Double.valueOf(teile[2]);
12 }
```

Merken:

- Wrapper: `Integer`, `Double`, `Boolean` können `null` sein.
- `isEmpty()`: prüft, ob zwischen den Trennzeichen überhaupt etwas steht.

7. Typischer Gesamt-Ablauf

1. Datei öffnen (Reader oder Writer), am besten mit `try-with-resources`.
2. Beim Schreiben: Zeilen/Text ausgeben (z.B. mit `println`, `printf`).
3. Beim Lesen: Zeilen mit `readLine()` einlesen.
4. Zeilen mit `split()` zerlegen und Werte parsen.
5. Fehler behandeln (`IOException`, `NumberFormatException`).